

UNIT 08

LESSON 08.02



Assigning Dynamic ID's to DOM Elements

Timing User Interaction with DOM Elements

In this lesson:

- We will use JS to dynamically assign id's and classes to DOM elements.
- We will then keep track of -- and time -- the user's hovering over these elements.
- We will then store the resulting data to an array of objects.

We do this to emulate software which records how long various page elements are holding user attention -- or are being ignored. Naturally, a real application would capture more than hovering; there's scrolling and clicking--and of course swiping on mobile devices. But for our purposes here, just tracking hovering will be a great coding exercise that lets us apply what we have just learned about **setInterval** while also giving us insight into how user engagement is actually tracked on the web.

1. Open **08.02-Timing-Using-Engagement.html** and preview the page in the browser. The user interface consists of three paragraphs and a button that says **Report Hover Time**. Notice that one of the paragraphs as an id, one has a class, and one has neither:

```
<p>
    This p tag has no attributes..
</p>

<p id="my-parag">
    This p tag has no class but it does have an id.
</p>

<p class="special-p">
    This p tag already the "special-p" class, but no id.
</p>
```

The idea is, the p tags have an irregular mix of id's and no id's. We will standardize them by giving them consecutively numbered ids.

2. Open the console side-by-side with the page elements.
3. Mouse over the first paragraph and hold the mouse there. Notice in the console that every second of hovering is being logged by a counter.
4. Mouse out (move the cursor away) from the first paragraph, and notice that a summary of the interaction appears in the console.

5. Repeat the mouseover-mouseout behavior on the other two paragraphs, pausing several seconds each time.
6. Click the button. Now, the console reports a summary of all three interactions: you get an array of objects, with each object being a record of an interaction, with properties of **elementID** and **hoverSeconds**:

```
{elementID: 'parag1', hoverSeconds: 3}
{elementID: 'parag2', hoverSeconds: 6}
{elementID: 'parag3', hoverSeconds: 4}
```

7. Now that we have seen the final product in action, switch from using **FINAL.js** to **PROG.js**.
8. We want to assign id's to all the p tags, so bring all three of them into JS at once with **querySelectorAll('p')**. This will make an array of objects:

```
const pTagArr = document.querySelectorAll('p');
```

9. Log the array to see what we have. It's the actual p-tags:

```
console.log('pTagArr:', pTagArr);
```

10. Loop through the array, assigning each item -- each paragraph object -- two listeners: one to call a function on **mouseover**, the other to call a function on **mouseout**. The function names are **onMouseOver** and **onMouseOut**, respectively:

```
for(let i = 0; i < pTagArr.length; i++) {
    pTagArr[i].addEventListener('mouseover', onMouseOver);
    pTagArr[i].addEventListener('mouseout', onMouseOut);
}
```

11. Assign the dynamic **id**, property, which starts with **parag** and concats on **(i+1)**, the loop counter, plus 1:

```
for(let i = 0; i < pTagArr.length; i++) {
    pTagArr[i].addEventListener('mouseover', onMouseOver);
    pTagArr[i].addEventListener('mouseout', onMouseOut);
    pTagArr[i].id = 'parag' + (i+1);
}
```

12. Assign the dynamic **className** property, which assign the existing **my-parag** class to the p tag element. Log the item each time through the loop:

```
for(let i = 0; i < pTagArr.length; i++) {
    pTagArr[i].addEventListener('mouseover', onMouseOver);
    pTagArr[i].addEventListener('mouseout', onMouseOut);
    pTagArr[i].id = 'parag' + (i+1);
    pTagArr[i].className = 'my-parag';
    console.log(pTagArr[i]);
}
```

13. Declare a few global variables:

- **myInterval** for stopping the **setInterval** method on mouseout
- **hoverSecs** for timing how many seconds a p tag is hovered over
- **reportArr** an array for storing the hover time data

```
let myInterval;
let hoverSecs = 0;
const reportArr = [];
```

14. Define the **onMouseOver** function. The first thing it does is instantiate a Date() object and call the getSeconds() method and save it to a variable called **startSec**. This returns the current time's seconds property. So if the time is now 1:23:45, **startSec** equals 45.

```
function onMouseOver() {
    let startSec = new Date().getSeconds();
}
```

15. We need to start a timer to see how long before the mouseout occurs, so call the **setInterval** method and pass it an arrow function for its callback. Set the whole expression equal to **myInterval**, so that later we can use the variable to clear the interval:

```
function onMouseOver() {
    let startSec = new Date().getSeconds();
    myInterval = setInterval(() => {
    });
}
```

16. Inside the callback function, declare a new instance of Date().getSeconds(), and instruct the callback to repeat every 1000 milliseconds:

```
function onMouseOver() {
    let startSec = new Date().getSeconds();
    myInterval = setInterval(() => {
        let newSec = new Date().getSeconds();
```

```
        }, 1000);
    }
```

17. Every 1 second, the callback function will run, with **newSec** updated each time. To count the mouseover time, set **hoverSecs** equal to **newSec** minus the original **startSec**:

```
    function onMouseOver() {
        let startSec = new Date().getSeconds();
        myInterval = setInterval(() => {
            let newSec = new Date().getSeconds();
            hoverSecs = newSec - startSec;
        }, 1000);
    }
```

18. Every minute the second counter reaches a maximum of 59 before restarting at 0. If this results in a negative **hoverSecs**, add 60 to the total. Log **hoverSecs** so that we see the hover seconds updating every second:

```
    function onMouseOver() {
        let startSec = new Date().getSeconds();
        myInterval = setInterval(() => {
            let newSec = new Date().getSeconds();
            hoverSecs = newSec - startSec;
            if(hoverSecs < 0) totSec += 60;
            console.log(hoverSecs);
        }, 1000);
    }
```

19. Declare the **onMouseOut** function, which runs when the user mouses out (moves away from) a p tag. The first thing to do is stop the counter, which we do by passing the **myInterval** variable to the **clearInterval** method:

```
    function onMouseOut() {
        clearInterval(myInterval);
    }
```

20. Log a message that tells how long the element of this particular id was hovered over:

```
    function onMouseOut() {
        clearInterval(myInterval);
        console.log(`element of id ${this.id} was hovered over for
${totSecs} seconds.`);
    }
```

21. Declare an object called **reportObj**, and assign it two properties:

- **elementID** set equal to **this.id**, the id of the tag
- **hoverSecs** set equal to the hoverSecs variable--the hover time

```
    function onMouseOut() {
        clearInterval(myInterval);
        console.log(`element of id ${this.id} was hovered over for
${totSecs} seconds.`);
        let reportObj = {
            elementID: this.id,
            hoverSecs: hoverSecs,
        }
    }
```

22. Push the object into the **reportArr** array:

```
    function onMouseOut() {
        clearInterval(myInterval);
        console.log(`element of id ${this.id} was hovered over for
${totSecs} seconds.`);
        let reportObj = {
            elementID: this.id,
            hoverSecs: hoverSecs,
        }
        reportArr.push(reportObj);
    }
```

23. Get the button:

```
    const reportBtn = document.querySelector('button');
```

24. Have the button listen for a click on itself and call the **reportHoverTime** function when a click occurs:

```
    reportBtn.addEventListener('click', reportHoverTime);
```

25. Write the **reportHoverTime** function. All it does is log the **reportArr** array to the console:

```
    function reportHoverTime() {
        console.log(reportArr);
    }
```

26. Refresh the browser and split the screen into web page and console.

27. Hover over the various p tags for several seconds each. Hover over the tags again, in a different order for varying amounts of time.

28. Click the **Report Hover Time** button. The object array with summary of hover data should appear in the console.

And with that we know how long the user hovered over each p tag.

END: Lesson 08.02